

Mini-Project 1: Risk, Flexibility, and Bias–Variance

EN.553.434/634: Elements of Statistical Learning

Jonathan Ma

Table of contents

1	Setup	2
2	1. Data-Generating Process	3
2.1	Q1. What is the Bayes predictor under squared-error loss?	4
2.2	Q2. What is the Bayes error?	4
3	2. Flexibility of kNN	5
3.1	Q3. Which value of K gives the most flexible fit? Why?	6
3.2	Q4. Briefly describe what changes as K increases.	6
4	3. Training and Test Error	7
4.1	Q5. Which K minimizes training error?	9
4.2	Q6. Which K approximately minimizes test error?	10
4.3	Q7. Why are these values generally different?	10
5	4. Changing the Training Set	10
5.1	Q8. Which value of K produces more variation across training sets?	12
6	5. Bias and Variance	12
6.1	Q9. What happens to bias and variance as flexibility increases?	15
6.2	Q10. Why do we need many independent training sets to estimate the variance above?	15
7	6. Bias–Variance Decomposition	15
7.1	Q11. Why should the two quantities agree only approximately in the simulation?	18

8	7. Takeaways	18
8.1	Q12. Why can a more flexible method have lower training error but higher test error?	18
8.2	Q13. In your own words, what does $\text{Var}_D(\hat{f}_D(x_0))$ measure?	18
8.3	Q14. What happens if the noise variance increases while f^* stays unchanged?	18
9	8. Optional Exploration: Curse of Dimensionality	19
9.1	Q15. What happens to the best test error as p increases?	22
9.2	Q16. Why can adding predictors that contain no information about Y hurt kNN?	22
9.3	Q17. How does this experiment connect to the curse of dimensionality?	22

1 Setup

We use the seed specified in the assignment to make the analysis reproducible.

```
set.seed(434634)

f_star <- function(x) {
  sin(2 * pi * x) + x
}

knn_predict <- function(X_train, y_train, X_new, K) {
  X_train <- as.matrix(X_train)
  X_new <- as.matrix(X_new)

  sapply(seq_len(nrow(X_new)), function(j) {
    d <- sqrt(
      rowSums(
        (X_train -
         matrix(
           X_new[j, ],
           nrow(X_train),
           ncol(X_train),
           byrow = TRUE
         ))^2
      )
    )

    neighbors <- order(d)[1:K]
    mean(y_train[neighbors])
  })
}
```

```

}

sigma <- 0.5
n <- 100

```

2 1. Data-Generating Process

The data-generating process is $X \sim \text{Unif}(-1, 1)$ and we are trying to find $Y = f^*(X) + \varepsilon$ where

$$f^*(x) = \sin(2\pi x) + x, \quad \varepsilon \sim N(0, 0.5^2).$$

We first generate one training set of size $n = 100$:

```

X_train <- runif(n, -1, 1)
y_train <- f_star(X_train) + rnorm(n, mean = 0, sd = sigma)

x_grid <- seq(-1, 1, length.out = 500)

plot(
  X_train,
  y_train,
  pch = 19,
  cex = 0.7,
  xlab = "X",
  ylab = "Y",
  main = "Training Data and True Regression Function"
)

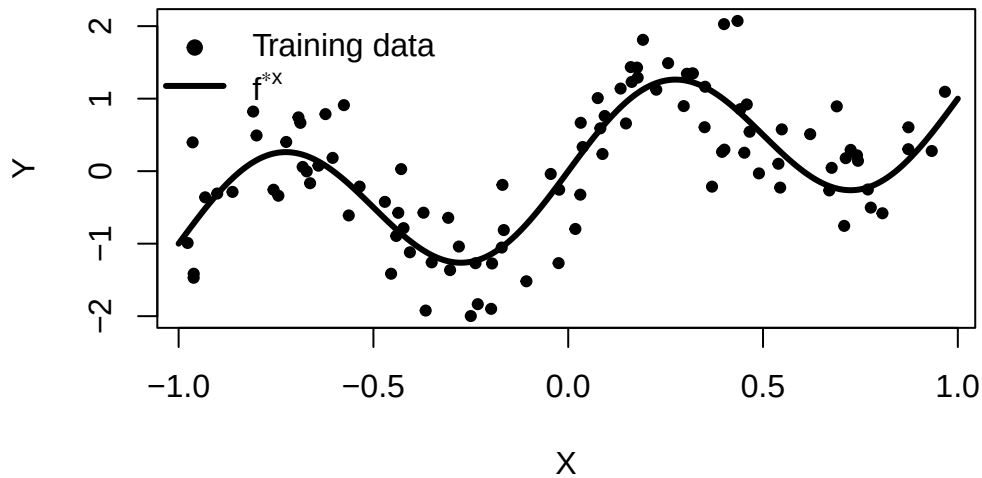
lines(
  x_grid,
  f_star(x_grid),
  lwd = 3
)

legend(
  "topleft",
  legend = c("Training data", expression(f~"*(x)")),
  pch = c(19, NA),
  lty = c(NA, 1),

```

```
lwd = c(NA, 3),
bty = "n"
)
```

Training Data and True Regression Function



2.1 Q1. What is the Bayes predictor under squared-error loss?

Under squared-error loss, the Bayes predictor is the conditional mean:

$$f^*(x) = E[Y \mid X = x] = \sin(2\pi x) + x.$$

Because the noise has conditional mean zero, the noise does not change the Bayes predictor.

2.2 Q2. What is the Bayes error?

The Bayes error under squared-error loss is the irreducible conditional variance:

$$R(f^*) = E[\text{Var}(Y \mid X)] = \sigma^2 = 0.5^2 = 0.25.$$

3 2. Flexibility of kNN

We fit k-nearest-neighbor regression for $K \in 1, 5, 20, 50$.

```
K_values <- c(1, 5, 20, 50)

par(mfrow = c(2, 2))

for (K in K_values) {

  pred_grid <- knn_predict(
    X_train,
    y_train,
    x_grid,
    K
  )

  plot(
    X_train,
    y_train,
    pch = 19,
    cex = 0.5,
    xlab = "X",
    ylab = "Y",
    main = paste("K =", K)
  )

  lines(
    x_grid,
    f_star(x_grid),
    lwd = 3
  )

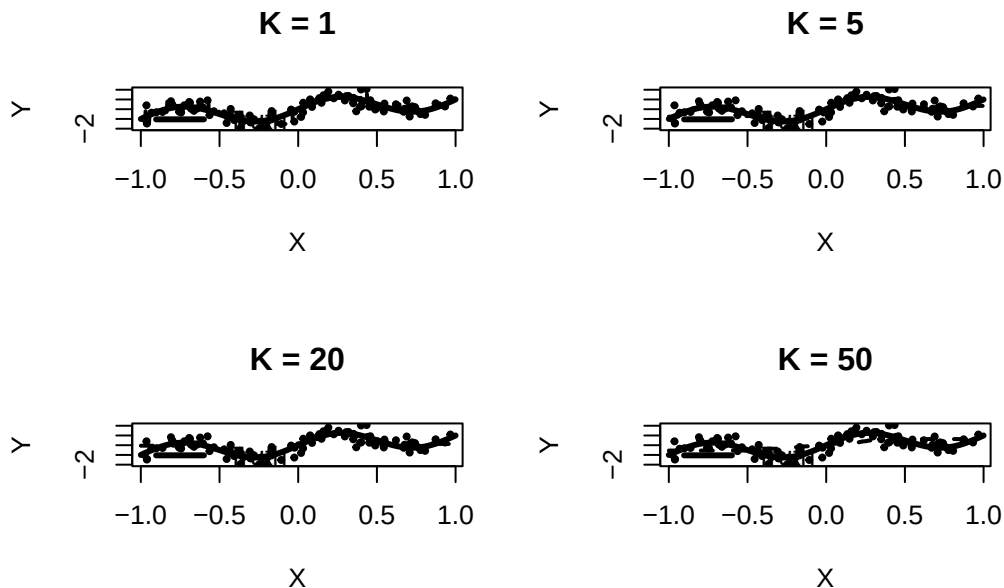
  lines(
    x_grid,
    pred_grid,
    lwd = 2,
    lty = 2
  )

  legend(
    "topleft",
```

```

legend = c("Truth", "kNN"),
lty = c(1, 2),
lwd = c(3, 2),
bty = "n"
)
}

```



```

par(mfrow = c(1, 1))

```

3.1 Q3. Which value of K gives the most flexible fit? Why?

$K = 1$ gives the most flexible fit because each prediction depends only on the single nearest training observation. Consequently, the fitted function can respond very strongly to local variation and noise in the training data.

3.2 Q4. Briefly describe what changes as K increases.

As K increases, each prediction averages over more observations, making the fitted function smoother and less flexible. Variance generally decreases while bias generally increases.

4 3. Training and Test Error

We generate an independent test set containing 5000 observations.

```
n_test <- 5000

X_test <- runif(n_test, -1, 1)
y_test <- f_star(X_test) +
  rnorm(n_test, mean = 0, sd = sigma)

K_grid <- 1:50

mse_train <- numeric(length(K_grid))
mse_test <- numeric(length(K_grid))

for (K in K_grid) {

  pred_train <- knn_predict(
    X_train,
    y_train,
    X_train,
    K
  )

  pred_test <- knn_predict(
    X_train,
    y_train,
    X_test,
    K
  )

  mse_train[K] <- mean(
    (y_train - pred_train)^2
  )

  mse_test[K] <- mean(
    (y_test - pred_test)^2
  )
}

best_train_K <- K_grid[which.min(mse_train)]
best_test_K <- K_grid[which.min(mse_test)]
```

```
best_train_K
```

```
[1] 1
```

```
best_test_K
```

```
[1] 11
```

```
min(mse_train)
```

```
[1] 0
```

```
min(mse_test)
```

```
[1] 0.2832302
```

```
ylim_error <- range(  
  c(mse_train, mse_test, sigma^2)  
)  
  
plot(  
  K_grid,  
  mse_test,  
  type = "l",  
  lwd = 2,  
  ylim = ylim_error,  
  xlab = "K",  
  ylab = "Mean Squared Error",  
  main = "Training and Test MSE versus K"  
)  
  
lines(  
  K_grid,  
  mse_train,  
  lwd = 2,  
  lty = 2  
)
```

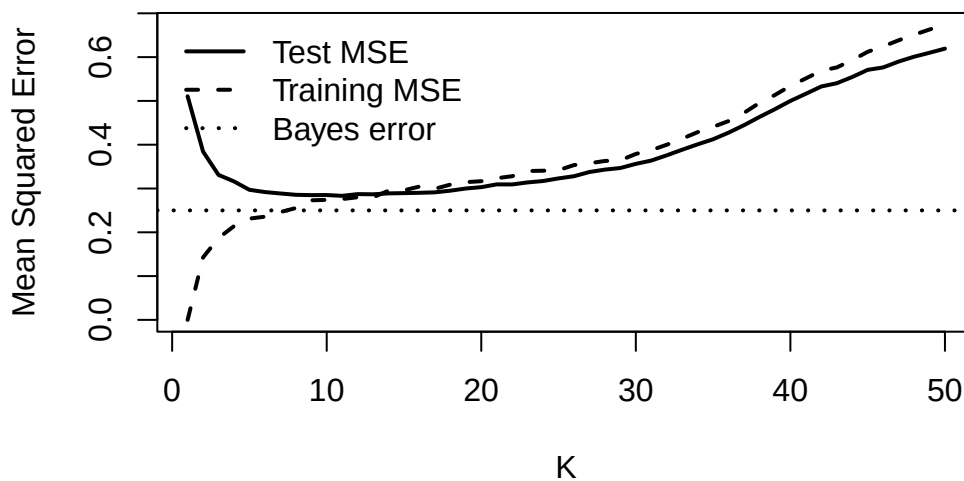
```

abline(
  h = sigma^2,
  lwd = 2,
  lty = 3
)

legend(
  "topleft",
  legend = c(
    "Test MSE",
    "Training MSE",
    "Bayes error"
  ),
  lty = c(1, 2, 3),
  lwd = 2,
  bty = "n"
)

```

Training and Test MSE versus K



As K increases, kNN becomes less flexible, because predictions average over increasingly large neighborhoods.

4.1 Q5. Which K minimizes training error?

The training error is minimized at $K = 1$.

In particular, $K = 1$ can interpolate the training responses because each training observation is its own nearest neighbor.

4.2 Q6. Which K approximately minimizes test error?

For this simulated training and test set, the test MSE is minimized at approximately $K = 11$, with minimum test MSE approximately 0.2832. The exact value can vary between simulated samples.

4.3 Q7. Why are these values generally different?

Training error rewards fitting the observed sample and therefore tends to favor highly flexible models. Test error measures performance on new observations, so excessive flexibility can fit training noise and increase generalization error.

5 4. Changing the Training Set

We compare a small value $K = 1$ with a relatively large value $K = 30$. For each value, we generate 10 independent training sets.

```
K_small <- 1
K_large <- 30
B_curves <- 10

fits_small <- matrix(
  NA,
  nrow = B_curves,
  ncol = length(x_grid)
)

fits_large <- matrix(
  NA,
  nrow = B_curves,
  ncol = length(x_grid)
)

for (b in 1:B_curves) {
  X_b <- runif(n, -1, 1)
```

```

y_b <- f_star(X_b) +
  rnorm(n, mean = 0, sd = sigma)

fits_small[b, ] <- knn_predict(
  X_b,
  y_b,
  x_grid,
  K_small
)

fits_large[b, ] <- knn_predict(
  X_b,
  y_b,
  x_grid,
  K_large
)
}

```

```

par(mfrow = c(1, 2))

matplot(
  x_grid,
  t(fits_small),
  type = "l",
  lty = 1,
  xlab = "X",
  ylab = "Predicted Y",
  main = paste("10 Training Sets: K =", K_small)
)

lines(
  x_grid,
  f_star(x_grid),
  lwd = 4
)

matplot(
  x_grid,
  t(fits_large),
  type = "l",
  lty = 1,
  xlab = "X",

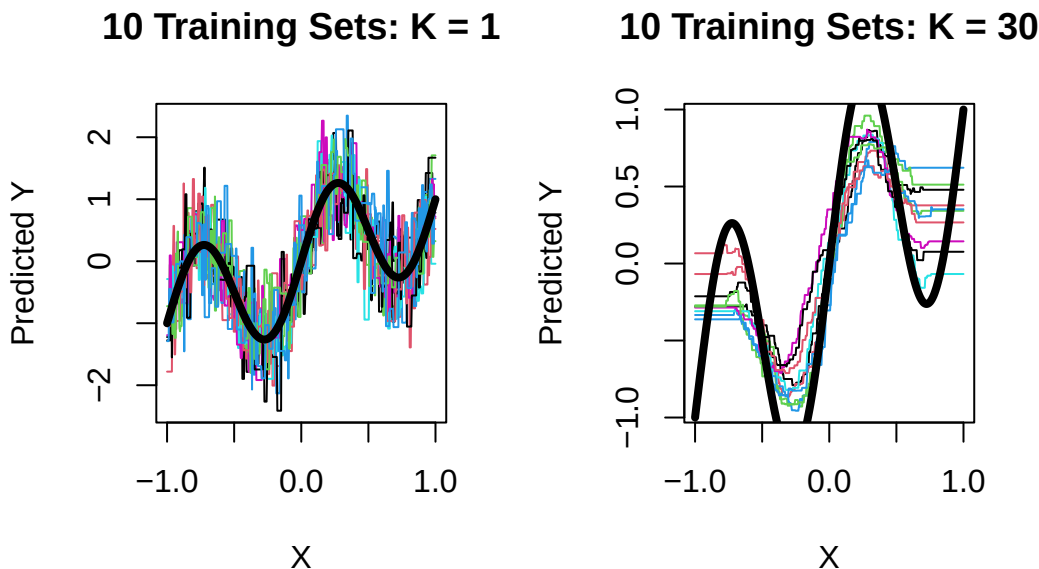
```

```

ylab = "Predicted Y",
main = paste("10 Training Sets: K =", K_large)
)

lines(
  x_grid,
  f_star(x_grid),
  lwd = 4
)

```



```

par(mfrow = c(1, 1))

```

5.1 Q8. Which value of K produces more variation across training sets?

The smaller value, $K = 1$, produces substantially more variation across training sets. Because each prediction depends on very few observations, small changes in the training sample can produce large changes in the fitted function.

6 5. Bias and Variance

Let $x_0 = 0.25$. We generate $B = 200$ independent training sets and record the kNN prediction at x_0 for every $K = 1, \dots, 50$.

```

x0 <- 0.25
B <- 200

true_x0 <- f_star(x0)

predictions <- matrix(
  NA,
  nrow = B,
  ncol = length(K_grid)
)

for (b in 1:B) {

  X_b <- runif(n, -1, 1)

  y_b <- f_star(X_b) +
    rnorm(n, mean = 0, sd = sigma)

  for (K in K_grid) {

    predictions[b, K] <- knn_predict(
      X_b,
      y_b,
      matrix(x0, nrow = 1),
      K
    )
  }
}

mean_prediction <- colMeans(predictions)

bias <- mean_prediction - true_x0
bias_squared <- bias^2

variance <- apply(
  predictions,
  2,
  var
)

plot(
  K_grid,

```

```

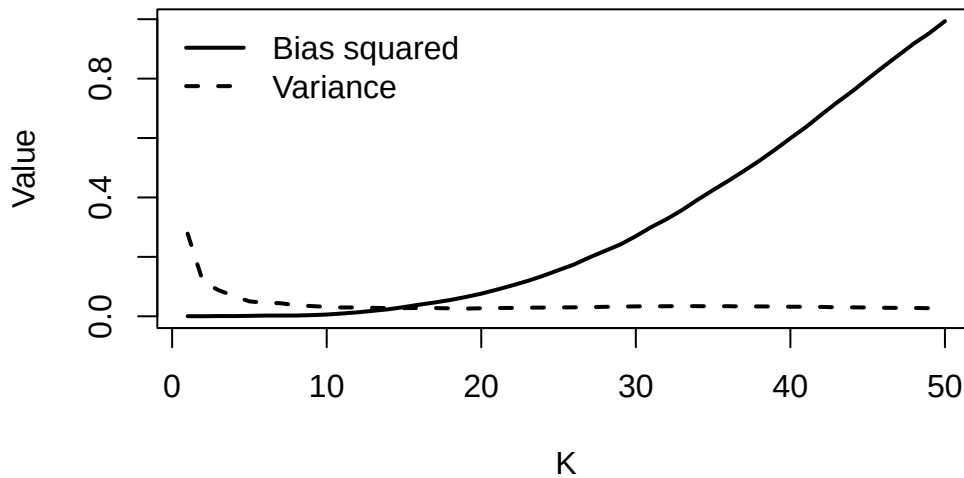
bias_squared,
type = "l",
lwd = 2,
ylim = range(c(bias_squared, variance)),
xlab = "K",
ylab = "Value",
main = "Estimated Bias Squared and Variance at x0 = 0.25"
)

lines(
  K_grid,
  variance,
  lwd = 2,
  lty = 2
)

legend(
  "topleft",
  legend = c("Bias squared", "Variance"),
  lty = c(1, 2),
  lwd = 2,
  bty = "n"
)

```

Estimated Bias Squared and Variance at $x_0 = 0.25$



6.1 Q9. What happens to bias and variance as flexibility increases?

For kNN, flexibility increases as K decreases. More flexible fits generally have lower bias but higher variance, while less flexible fits generally have higher bias but lower variance.

6.2 Q10. Why do we need many independent training sets to estimate the variance above?

The variance measures how much $\hat{f}_D(x_0)$ changes across different possible training sets D . A single training set provides only one fitted value, so repeated independent samples are needed to estimate this sampling variability.

7 6. Bias–Variance Decomposition

For each fitted model and training set, we independently generate $M = 1000$ future responses at x_0 and average the resulting squared prediction errors.

$$Y_0 = f^*(x_0) + \varepsilon_0, \quad \varepsilon_0 \sim N(0, 0.5^2).$$

We then compare the empirical squared prediction error with:

$$\sigma^2 + \widehat{\text{Bias}}^2 + \widehat{\text{Var}}.$$

```
# Number of independent future responses generated
# for each fitted model
M <- 1000

estimated_prediction_risk <- numeric(length(K_grid))

for (K in K_grid) {

  squared_errors <- numeric(B * M)

  for (b in 1:B) {

    # Generate M independent future responses at x0
    y0 <- true_x0 +
      rnorm(M, mean = 0, sd = sigma)
```

```

# Prediction from training set b using this K
squared_errors[((b - 1) * M + 1):(b * M)] <-
  (y0 - predictions[b, K])^2
}

# Average over both training sets and future responses
estimated_prediction_risk[K] <-
  mean(squared_errors)
}

# Bias-variance decomposition
bv_decomposition <-
  sigma^2 + bias_squared + variance

comparison <- data.frame(
  K = K_grid,
  Prediction_Risk = estimated_prediction_risk,
  Noise_Bias2_Variance = bv_decomposition
)

head(comparison)

```

	K	Prediction_Risk	Noise_Bias2_Variance
1	1	0.5283521	0.5283211
2	2	0.3654799	0.3650345
3	3	0.3370832	0.3385934
4	4	0.3173988	0.3182408
5	5	0.3002162	0.3014561
6	6	0.2983171	0.2978012

A graphical comparison is:

```

plot(
  K_grid,
  estimated_prediction_risk,
  type = "l",
  lwd = 2,
  xlab = "K",
  ylab = "Squared Error",
  ylim = range(
    c(

```

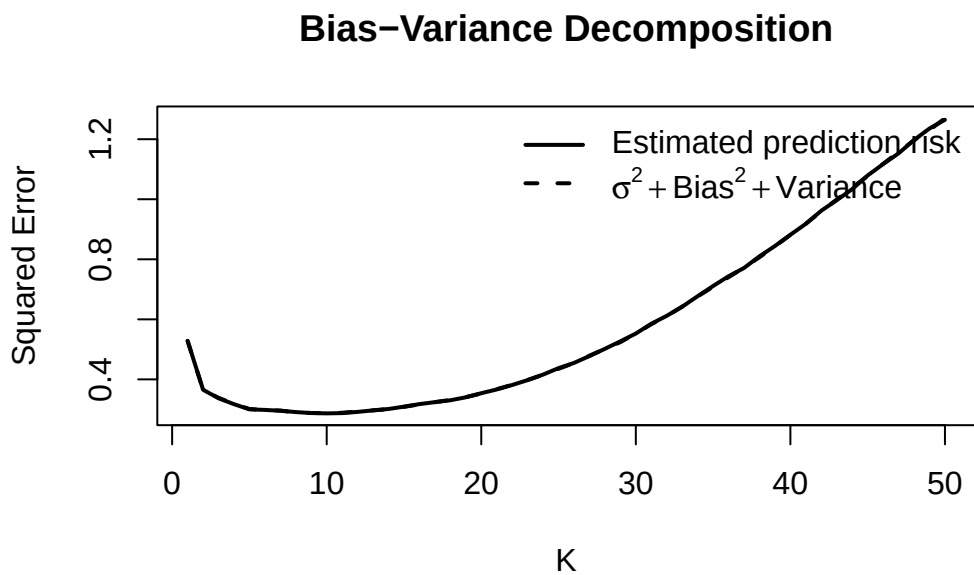
```

    estimated_prediction_risk,
    bv_decomposition
  )
),
main = "Bias-Variance Decomposition"
)

lines(
  K_grid,
  bv_decomposition,
  lwd = 2,
  lty = 2
)

legend(
  "topright",
  legend = c(
    "Estimated prediction risk",
    expression(sigma^2 + Bias^2 + Variance)
  ),
  lty = c(1, 2),
  lwd = 2,
  bty = "n"
)

```



For a compact numerical comparison, we display several representative values of K :

```
selected_K <- c(1, 5, 10, 20, 50)
```

```
comparison[selected_K, ]
```

	K	Prediction_Risk	Noise_Bias2_Variance
1	1	0.5283521	0.5283211
5	5	0.3002162	0.3014561
10	10	0.2867295	0.2874373
20	20	0.3539391	0.3529890
50	50	1.2647933	1.2697537

7.1 Q11. Why should the two quantities agree only approximately in the simulation?

The two quantities agree only approximately because bias, variance, and prediction risk are estimated from a finite number of independently simulated training sets and test responses. Thus, Monte Carlo sampling error remains. In addition, the sample variance uses a finite-sample $B - 1$ denominator, so the empirical quantities need not satisfy the population decomposition exactly.

8 7. Takeaways

8.1 Q12. Why can a more flexible method have lower training error but higher test error?

A highly flexible method can adapt closely to the observed training data, including random noise, which lowers training error. This overfitting increases sensitivity to the particular training sample and can therefore increase error on new observations.

8.2 Q13. In your own words, what does $\text{Var}_D(\hat{f}_D(x_0))$ measure?

It measures how much the fitted prediction at the same point x_0 would change if we repeatedly collected new training datasets from the same population and refit the model.

8.3 Q14. What happens if the noise variance increases while f^* stays unchanged?

The Bayes predictor remains $f^*(x) = E[Y | X = x]$, so it does not change when only the noise variance increases. However, the Bayes error increases because the irreducible prediction error equals the conditional noise variance.

9 8. Optional Exploration: Curse of Dimensionality

We now add predictors that contain no information about the response. Only X_1 determines Y :

$$Y = f^*(X_1) + \varepsilon.$$

We compare:

$$p \in 1, 2, 5, 10$$

```
dimensions <- c(1, 2, 5, 10)

test_mse_dimension <- matrix(
  NA,
  nrow = length(dimensions),
  ncol = length(K_grid)
)

for (j in seq_along(dimensions)) {

  p <- dimensions[j]

  X_train_p <- matrix(
    runif(n * p, -1, 1),
    nrow = n,
    ncol = p
  )

  y_train_p <- f_star(X_train_p[, 1]) +
    rnorm(n, mean = 0, sd = sigma)

  X_test_p <- matrix(
    runif(n_test * p, -1, 1),
    nrow = n_test,
    ncol = p
  )

  y_test_p <- f_star(X_test_p[, 1]) +
    rnorm(n_test, mean = 0, sd = sigma)
```

```

for (K in K_grid) {

  pred_test_p <- knn_predict(
    X_train_p,
    y_train_p,
    X_test_p,
    K
  )

  test_mse_dimension[j, K] <-
    mean((y_test_p - pred_test_p)^2)
}
}

```

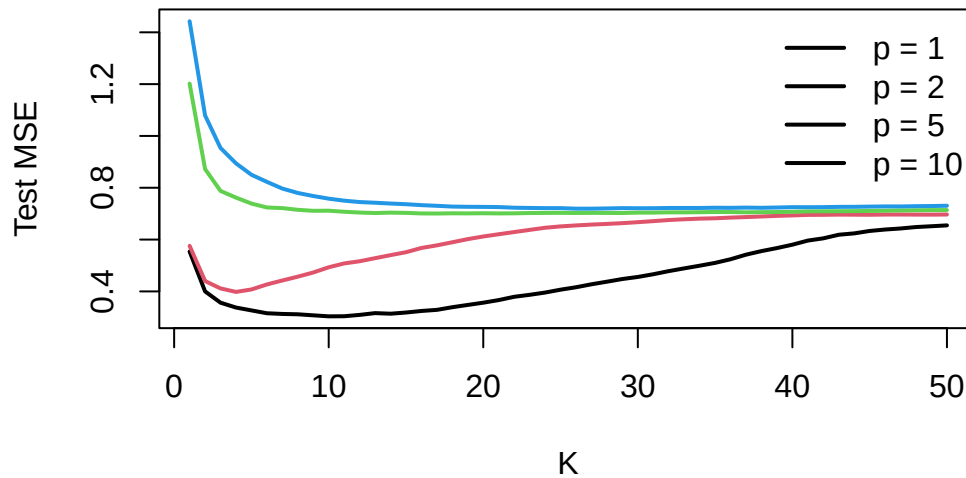
```

matplot(
  K_grid,
  t(test_mse_dimension),
  type = "l",
  lty = 1,
  lwd = 2,
  xlab = "K",
  ylab = "Test MSE",
  main = "Curse of Dimensionality in kNN"
)

legend(
  "topright",
  legend = paste("p =", dimensions),
  lty = 1,
  lwd = 2,
  bty = "n"
)

```

Curse of Dimensionality in kNN



The minimum test MSE for each dimension is:

```
minimum_mse <- apply(
  test_mse_dimension,
  1,
  min
)

best_K_dimension <- apply(
  test_mse_dimension,
  1,
  which.min
)

dimension_results <- data.frame(
  p = dimensions,
  Best_K = best_K_dimension,
  Minimum_Test_MSE = minimum_mse
)

dimension_results
```

	p	Best_K	Minimum_Test_MSE
1	1	10	0.3036916
2	2	4	0.3980303
3	5	17	0.7006228

9.1 Q15. What happens to the best test error as p increases?

In this simulation, the minimum achievable test error generally increases as additional irrelevant dimensions are added. The exact values depend on the simulated samples, but the overall deterioration illustrates the difficulty kNN faces in higher-dimensional spaces.

In this simulation, the minimum test MSE increases substantially as irrelevant dimensions are added, although it is not strictly monotonic because of simulation variability. The deterioration from $p = 1$ to $p = 5$ and $p = 10$ illustrates the curse of dimensionality for kNN.

9.2 Q16. Why can adding predictors that contain no information about Y hurt kNN?

kNN determines neighbors using distance across all p predictors. Because the additional predictors contain no information about Y , differences along those coordinates contribute noise to the distance metric. Consequently, points that are close in the informative variable X_1 may no longer be selected as nearest neighbors, degrading the quality of the local average.

9.3 Q17. How does this experiment connect to the curse of dimensionality?

As dimensionality increases, observations become increasingly sparse in the feature space and meaningful local neighborhoods become harder to identify. Consequently, kNN requires substantially more data to find observations that are genuinely close to a target point.